

CS 201, Spring 2026

Homework Assignment 4

Due: 23:59, May 8, 2026

1 Introduction

Welcome to the Asteroid Belt System!

In this assignment, you will design a simulation system to model interactions within a one-dimensional asteroid field.

Each asteroid in the field is characterized by:

- a unique **ID**,
- a positive **energy** value,
- and a **direction**:
 - R: moving to the right,
 - L: moving to the left.

Asteroids move along a single line. When two asteroids moving toward each other meet, they collide according to well-defined rules. Your system will manage these asteroids, simulate collisions, and allow dynamic updates to the field.

Your tasks include:

- maintaining the asteroid field,
- simulating collisions until the system becomes stable,
- dynamically adding new asteroids,
- reversing the direction of the rightmost asteroid,
- reversing all asteroids in the field,
- and answering queries about asteroid status.

1.1 Input Specifications

We will provide a file named `asteroidFile`, which contains the following information:

- The first line contains a single integer `n`, representing the number of asteroids initially in the field.
- Each of the next `n` lines contains three values in the following format:

<id> <energy> <direction>

where:

- `id` is the unique ID of the asteroid,
- `energy` is a positive integer,
- `direction` is either `R` or `L`.

For example, an `asteroidFile` may look like this:

```
1 6
2 1 5 R
3 2 8 R
4 3 3 L
5 4 6 L
6 5 4 R
7 6 7 L
```

This means that the asteroid field initially contains 6 asteroids in the given order from left to right.

1.2 Collision Rules

All asteroids move at the same speed along a one-dimensional line. The position of an asteroid corresponds to its index in the field: the first asteroid in the input is the leftmost, and the most recently added asteroid is the rightmost.

- Asteroids moving in the same direction will never meet.
- A collision may occur only when two asteroids are moving toward each other.
- If two asteroids collide and have equal energy, both are destroyed.
- Otherwise, the asteroid with higher energy survives, and its energy is reduced by the energy of the other asteroid.

A surviving asteroid continues moving in the same direction and may collide again later.

1.3 Class Implementation

Your solution must be implemented in a class called **AsteroidField**. Below is the required public part of the **AsteroidField** class. The interface for the class must be written in a file called **AsteroidField.h** and its implementation must be written in a file called **AsteroidField.cpp**. You can define additional public and private member functions and data members in this class. You can also define additional classes in your solution.

```

1 class AsteroidField {
2
3 public:
4     AsteroidField(const string asteroidFile);
5     ~AsteroidField();
6
7     void display() const;
8     void collide();
9
10    void addAsteroid(const int id, const int energy, const char direction);
11    void reverseLastAsteroid();
12    void reverseField();
13
14    void displayAsteroidStatus(const int id) const;
15
16 };

```

The member functions are defined as follows:

- **AsteroidField** Constructor. Reads the asteroid information from the **asteroidFile** and stores the current asteroid field in the given order. No collision simulation is performed automatically in the constructor.

- **display** Displays the current asteroid field from left to right.

- If the field is empty, it should display:

```

1 Asteroid field is empty.

```

- **collide** Simulates all possible collisions in the current asteroid field until the field becomes stable.

- This function must display:

```

1 Collision simulation complete.

```

- **addAsteroid** Adds a new asteroid to the **end** of the current field.

- This function must display:

```

1 Asteroid A<X> added.

```

where X is the ID of the asteroid.

- **reverseLastAsteroid** Reverses the **direction** of the **rightmost** asteroid.

- If successful, display:

```
1 Asteroid A<X> reversed.
```

– If the field is empty, display:

```
1 No asteroid to reverse.
```

- **reverseField** Reverses the **order** of all asteroids in the field.

- The directions and energies of asteroids remain unchanged.
- The leftmost asteroid becomes the rightmost, and vice versa.
- If the field is empty, display:

```
1 Asteroid field is empty.
```

– Otherwise, display:

```
1 Asteroid field order reversed.
```

- **displayAsteroidStatus** Displays the status of the asteroid with the given ID.

– If active:

```
1 Asteroid A<X> is active with energy E and direction D.
```

– If destroyed:

```
1 Asteroid A<X> is destroyed.
```

– If it does not exist:

```
1 Asteroid A<X> does not exist.
```

1.4 Example Test Case and Output

This section walks through an example test case and the corresponding output. We will use a similar program to test your solution, so make sure that the name of the class is **AsteroidField**, its interface is in the file called **AsteroidField.h**, and the required functions are defined as shown above.

Input File: asteroidFile.txt

```
1 6
2 1 5 R
3 2 8 R
4 3 3 L
5 4 6 L
6 5 4 R
7 6 7 L
```

Above input represents an asteroid field with 6 asteroids stored from left to right in the given order.

Example Test Program

```
1 #include <iostream>
2 #include "AsteroidField.h"
3
4 using namespace std;
5
6 int main()
7 {
8     AsteroidField af("asteroidFile.txt");
9
10    af.display();
11    cout << endl;
12
13    af.collide();
14    af.display();
15    cout << endl;
16
17    af.displayAsteroidStatus(1);
18    af.displayAsteroidStatus(2);
19    af.displayAsteroidStatus(10);
20    cout << endl;
21
22    af.reverseLastAsteroid();
23    af.display();
24    cout << endl;
25
26    af.collide();
27    af.display();
28    cout << endl;
29
30    af.addAsteroid(7, 3, 'L');
31    af.addAsteroid(8, 5, 'R');
32    af.display();
33    cout << endl;
34
35    af.collide();
36    af.display();
37    cout << endl;
38
39    af.reverseLastAsteroid();
40    af.display();
41    cout << endl;
42
43    af.collide();
```

```

44     af.display();
45     cout << endl;
46
47     af.displayAsteroidStatus(7);
48     af.displayAsteroidStatus(8);
49     cout << endl;
50
51     af.addAsteroid(9, 4, 'R');
52     af.addAsteroid(10, 2, 'L');
53     af.display();
54     cout << endl;
55
56     af.collide();
57     af.display();
58     cout << endl;
59
60     af.displayAsteroidStatus(9);
61     af.displayAsteroidStatus(10);
62     cout << endl;
63
64     af.reverseField();
65     af.display();
66     cout << endl;
67
68     af.collide();
69     af.display();
70
71     return 0;
72 }

```

Expected Output

```

1 Current asteroid field:
2 A1(5,R) A2(8,R) A3(3,L) A4(6,L) A5(4,R) A6(7,L)
3
4 Collision simulation complete.
5 Current asteroid field:
6 A1(1,R)
7
8 Asteroid A1 is active with energy 1 and direction R.
9 Asteroid A2 is destroyed.
10 Asteroid 10 does not exist.
11
12 Asteroid A1 reversed.
13 Current asteroid field:
14 A1(1,L)
15
16 Collision simulation complete.

```

```

17 Current asteroid field:
18 A1(1,L)
19
20 Asteroid A7 added.
21 Asteroid A8 added.
22 Current asteroid field:
23 A1(1,L) A7(3,L) A8(5,R)
24
25 Collision simulation complete.
26 Current asteroid field:
27 A1(1,L) A7(3,L) A8(5,R)
28
29 Asteroid A8 reversed.
30 Current asteroid field:
31 A1(1,L) A7(3,L) A8(5,L)
32
33 Collision simulation complete.
34 Current asteroid field:
35 A1(1,L) A7(3,L) A8(5,L)
36
37 Asteroid A7 is active with energy 3 and direction L.
38 Asteroid A8 is active with energy 5 and direction L.
39
40 Asteroid A9 added.
41 Asteroid A10 added.
42 Current asteroid field:
43 A1(1,L) A7(3,L) A8(5,L) A9(4,R) A10(2,L)
44
45 Collision simulation complete.
46 Current asteroid field:
47 A1(1,L) A7(3,L) A8(5,L) A9(2,R)
48
49 Asteroid A9 is active with energy 2 and direction R.
50 Asteroid A10 is destroyed.
51
52 Asteroid field order reversed.
53 Current asteroid field:
54 A9(2,R) A8(5,L) A7(3,L) A1(1,L)
55
56 Collision simulation complete.
57 Current asteroid field:
58 A8(3,L) A7(3,L) A1(1,L)

```

2 Specifications

1. You ARE NOT ALLOWED to modify the given parts of the header file.
2. You MUST use a nonrecursive solution using a stack as the main data structure for

implementing the collision simulation in `collide()`. You will get no points if your main collision logic is not stack-based.

3. You **MUST** implement your own stack. You may use the stack implementations discussed during the lectures as a reference, but you must write your own code. You **ARE NOT ALLOWED** to use STL containers such as `stack`, `vector`, `list`, or any other external library container implementations. You are also **NOT ALLOWED** to implement your own list or array-based container to simulate the collision behavior. The collision logic must be performed using your stack.
4. You may implement additional data structures such as a list to store asteroid objects and track their status.
5. Moreover, you **ARE NOT ALLOWED** to use any global variables or any global functions.
6. Output message for each operation **MUST** match the format shown in the output of the example code.
7. Your code **MUST NOT** have any memory leaks. You will lose points if you have memory leaks in your program even though the outputs of the operations are correct. To detect memory leaks, you may want to use Valgrind which is available at <http://valgrind.org>.
8. You may assume that all asteroid IDs given in the input file are unique. Similarly, every call to `addAsteroid()` will use a new ID that does not already exist in the system.
9. A call to `collide()` should always simulate collisions on the **current** asteroid field, taking into account any asteroids added later by `addAsteroid()` and any direction changes caused by `reverseLastAsteroid()`.
10. The functions `addAsteroid()`, `reverseLastAsteroid()` and `reverseField()` do **not** automatically simulate collisions. Any new collision opportunities created by these operations must be resolved only when `collide()` is called.

3 Submission

1. In this assignment, you must have separate interface and implementation files (i.e., separate `.h` and `.cpp` files) for your class. Your class name **MUST BE** `AsteroidField` and your file names **MUST BE** `AsteroidField.h` and `AsteroidField.cpp`. Note that you may write additional class(es) in your solution.
2. The code (main function) given above is just an example. We will test your implementation using different scenarios, which will contain different function calls. Thus, do not test your implementation only by using this example code. We recommend you to write your own driver files to make extra tests. However, you **MUST**

NOT submit these test codes (we will use our own test code). In other words, do not submit a file that contains a function called `main`.

3. You should put all of your `.h` and `.cpp` files into a folder and zip the folder (in this zip file, there should not be any file containing a `main` function). The name of this zip file should conform to the following name convention:

`secX-Firstname-Lastname-StudentID.zip`

where X is your section number. The submissions that do not obey these rules will not be graded. Please do not use Turkish letters in your file and folder names.

4. Make sure that each file that you submit (each and every file in the archive) contains your name, section, and student number at the top as comments.
5. You are free to write your programs in any environment (you may use Linux, Windows, MacOS, etc.). On the other hand, we will test your programs on `dijkstra.ug.bcc.bilkent.edu.tr` and we will expect your programs to compile and run on the dijkstra machine. If we could not get your program to properly work on the dijkstra machine, you would lose a considerable amount of points. Thus, we recommend you to make sure that your program compiles and properly works on `dijkstra.ug.bcc.bilkent.edu.tr` before submitting your assignment.
6. This assignment is due by 23:59 on May 8, 2026. You should upload your work to Moodle before the deadline. No hardcopy submission is needed. Late submissions will not be accepted (if you can upload to Moodle, then you are fine). There will be no extension to this deadline.
7. We use an automated tool as well as manual inspection to check your submissions against plagiarism. For questions regarding academic integrity and use of external tools (including generative AI tools), please refer to the course home page and the Honor Code for Introductory Programming Courses (CS 101/102/201/202) at https://docs.google.com/document/d/1v_3ltpV_1ClLsROXrMbojyuv4KrFQAm1uoz3SdC-7es/edit?usp=sharing.
8. This homework will be graded by your TA **Sude Önder** (`sude.onder@bilkent.edu.tr`). Thus, you may ask your homework-related questions directly to her. There will also be a forum on Moodle for questions.